

**Practical File submitted for the
evaluation of
“Big Data Analytics,
Tools & Techniques
Level-III Lab”
Subject Code: 21CS0302**

**MCA – Second Year (IV Semester)
Department of Computer Science**



Submitted By:

Name: Sahil Gupta

Roll No: 40824210020

Specialization: Computer Science

Submitted To:

Dr. Priyanka Maan

INDEX

S.No.	List of Programs	Date	Remarks
1.	Introduction to Big Data Module. To study big data and its type with its applications.	22/01/ 2026	
2.	Web Scraping and extract all hyperlinks (<a> tags) from the BrainyQuote website along with their corresponding text.	29/01/ 2026	
3.	Introduction to Hadoop Module.	05/02/ 2026	
4.	Introduction to Map Reduce Module.	12/02/ 2026	
5.	Write a program to perform matrix multiplication using Map Reduce Technique.	19/02/ 2026	
6.	Write a program to perform matrix multiplication using Pyspark Library in Python.	19/02/ 2026	
7.	Write a program to implement wordcount using Map Reduce.	05/03/ 2026	
8.	Installation of VMware virtual box.	12/03/ 2026	
9.	To execute any 10 Hadoop commands on VmWare.	19/03/ 2026	
10.	Introduction to Hive and pig data module.	19/03/ 2026	
11.	Write a program to read and store the data using PIG grunt shell. Execute the diagnostic operators on this data.	02/04/2026	
12.	Write any 10 HIVE DDL commands and HIVE DML commands.	16/04/2026	
13.	Write a HQL command to create a table employee with your own attribute and execute the following command • SELECT all fields • GROUP BY department • WHERE any condition • HAVING • ORDER BY employee number	16/04/2026	
14.	To perform data analysis using Sqoop.	23/04/2026	

Practical - 8

Aim:- Installation of VMware virtual box.

VMware Workstation Player

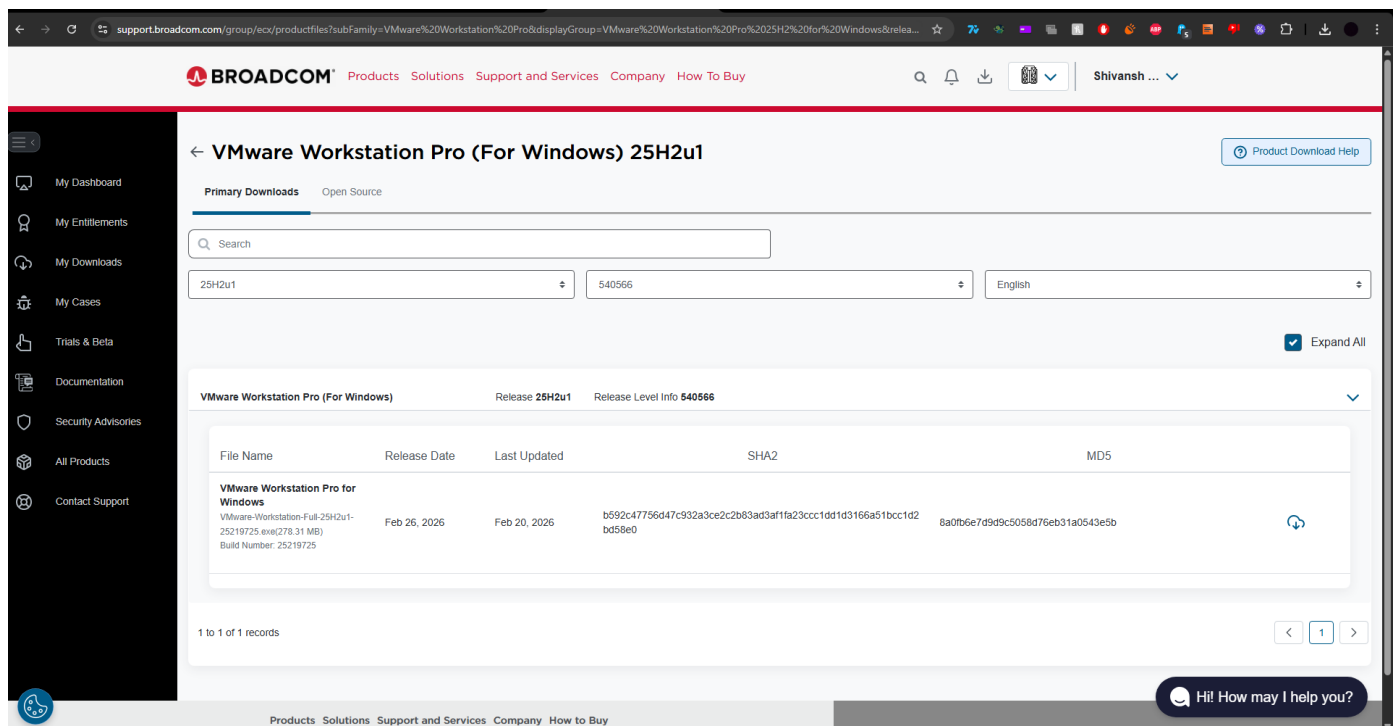
VMware Workstation Player is a virtualization tool that enables users to run multiple operating systems on a single computer. It creates virtual machines (VMs) that function like real systems, making it useful for testing, development, and learning without affecting the main system.

System Requirements

- 64-bit Windows 10/11
- Minimum 4 GB RAM (8 GB recommended)
- Virtualization enabled (Intel VT-x / AMD-V)
- At least 20 GB free disk space

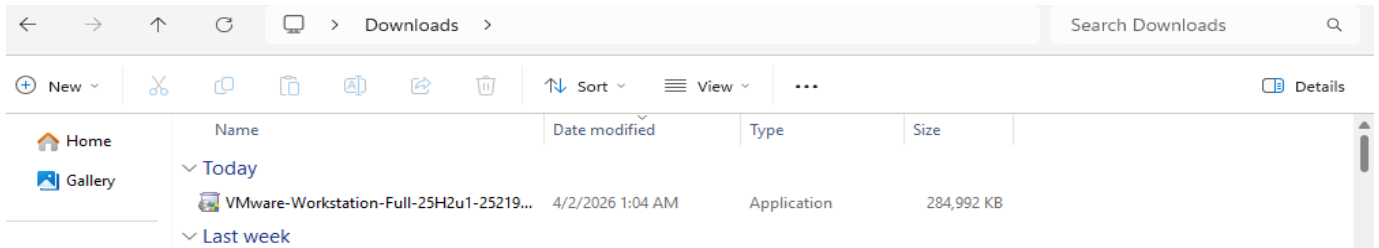
1. Download VMware Workstation Player

- Open a web browser and go to the official VMware website
- Navigate to the **Downloads** section
- Select **Workstation Player**
- Click on **Download Now** and choose the Windows version



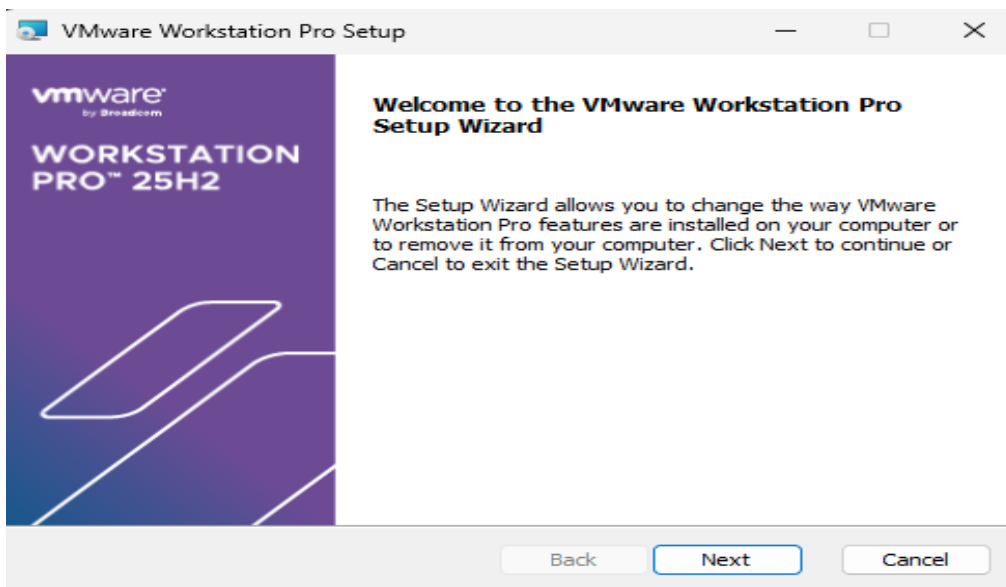
2. Run the Installer

- Locate the downloaded .exe file in your system
- Double-click on it to start installation
- Click **Yes** if permission is requested



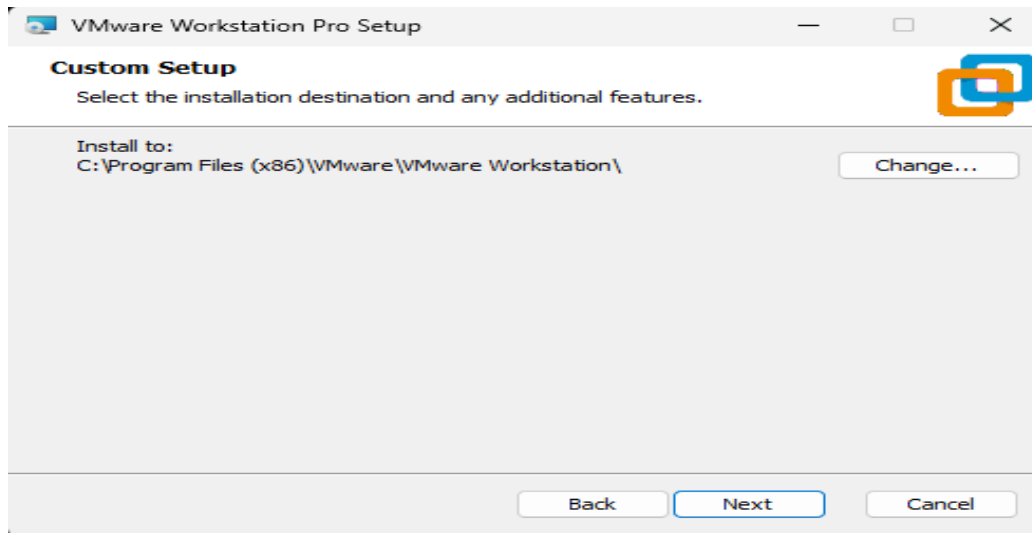
3. Start Installation Wizard

- The setup wizard will open
- Click **Next** to proceed
- Accept the **License Agreement**
- Click **Next** again



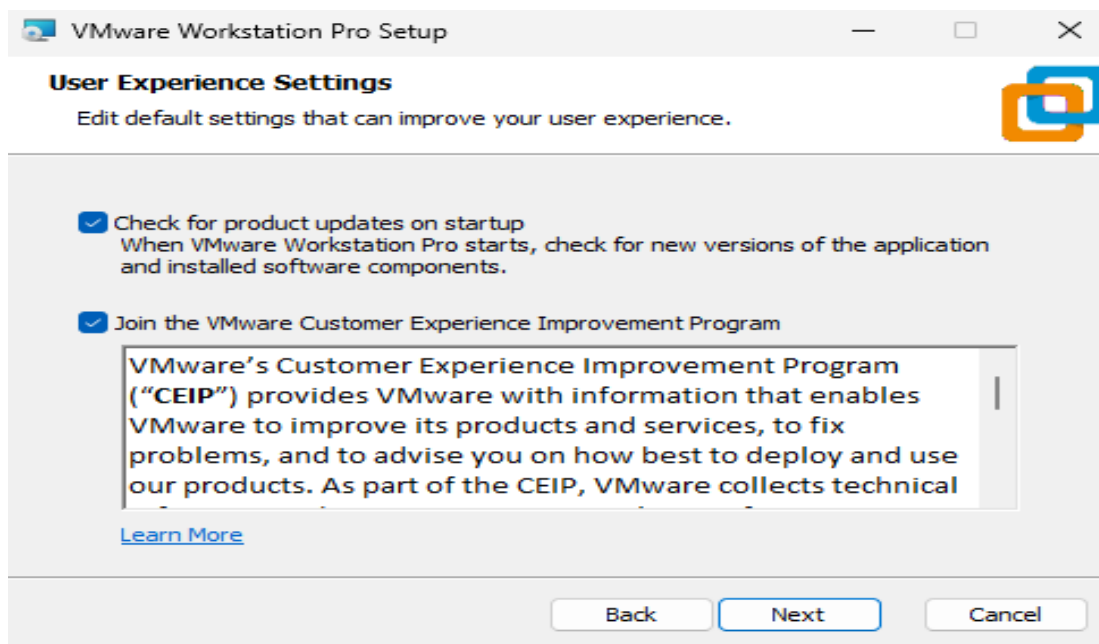
4. Choose Installation Options

- Select installation location (default is recommended)
- Optionally enable:
 - Enhanced Keyboard Driver
 - Add VMware to system PATH
- Click **Next**



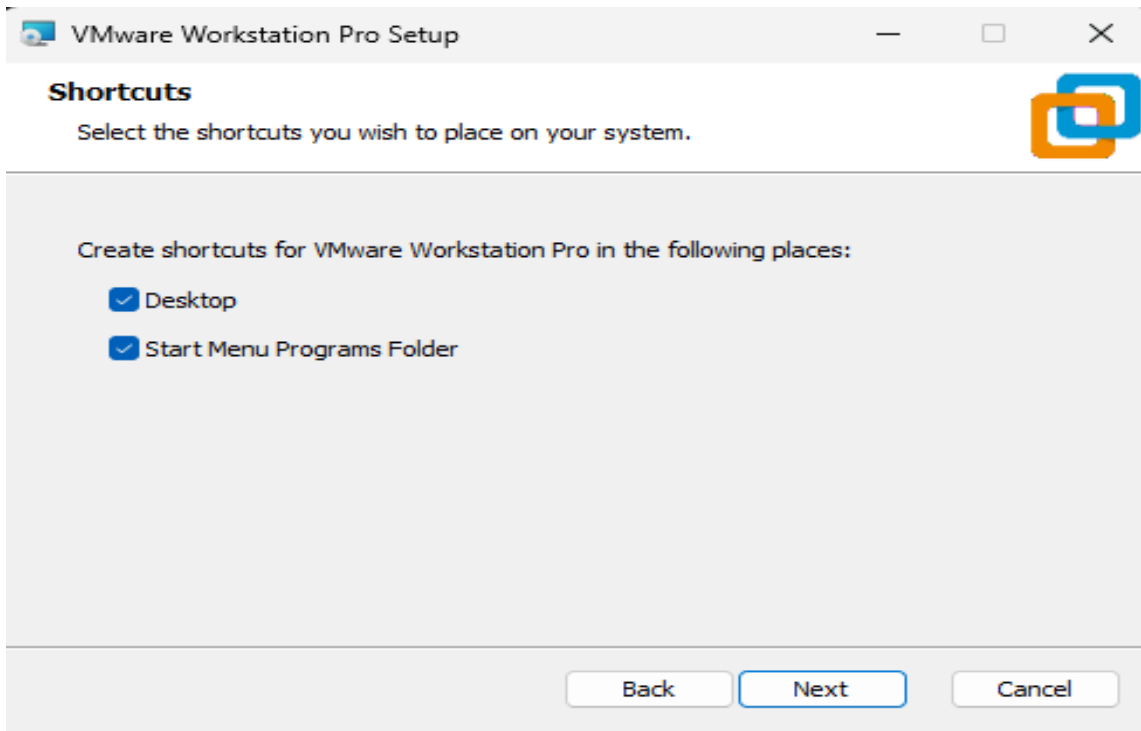
5. Configure User Experience Settings

- Choose whether to join the Customer Experience Improvement Program
- This step is optional
- Click **Next**



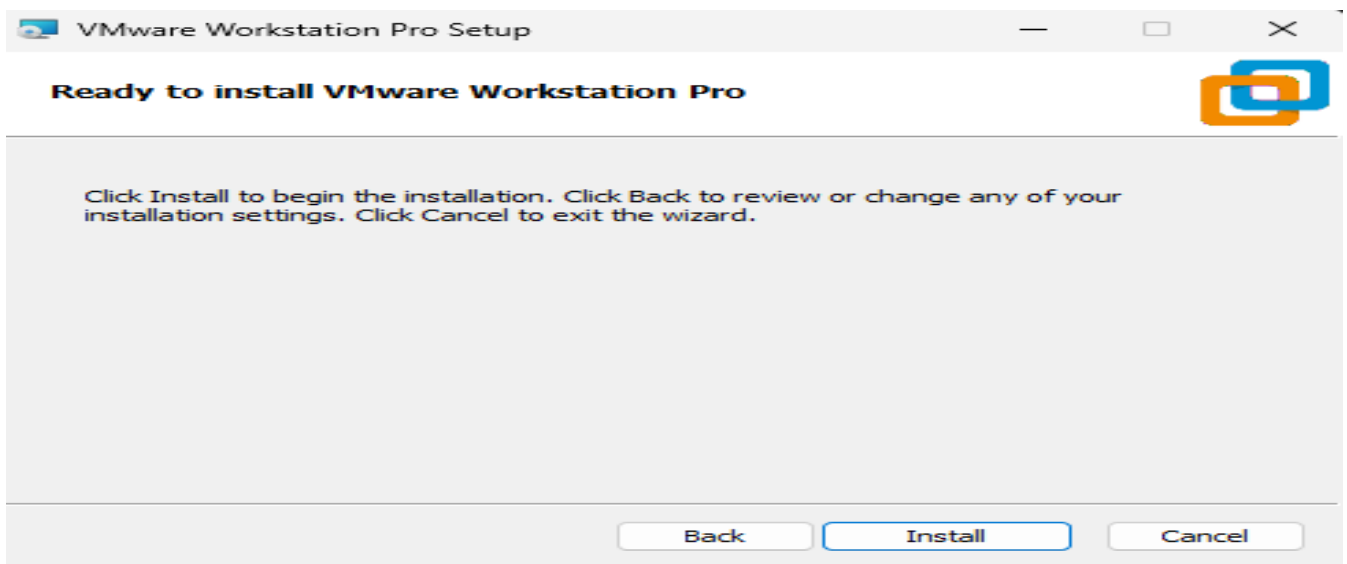
6. Create Shortcuts

- Select where you want shortcuts (Desktop / Start Menu)
- Click **Next**



7. Install VMware

- Click on **Install** button
- Wait for installation to complete (may take a few minutes)

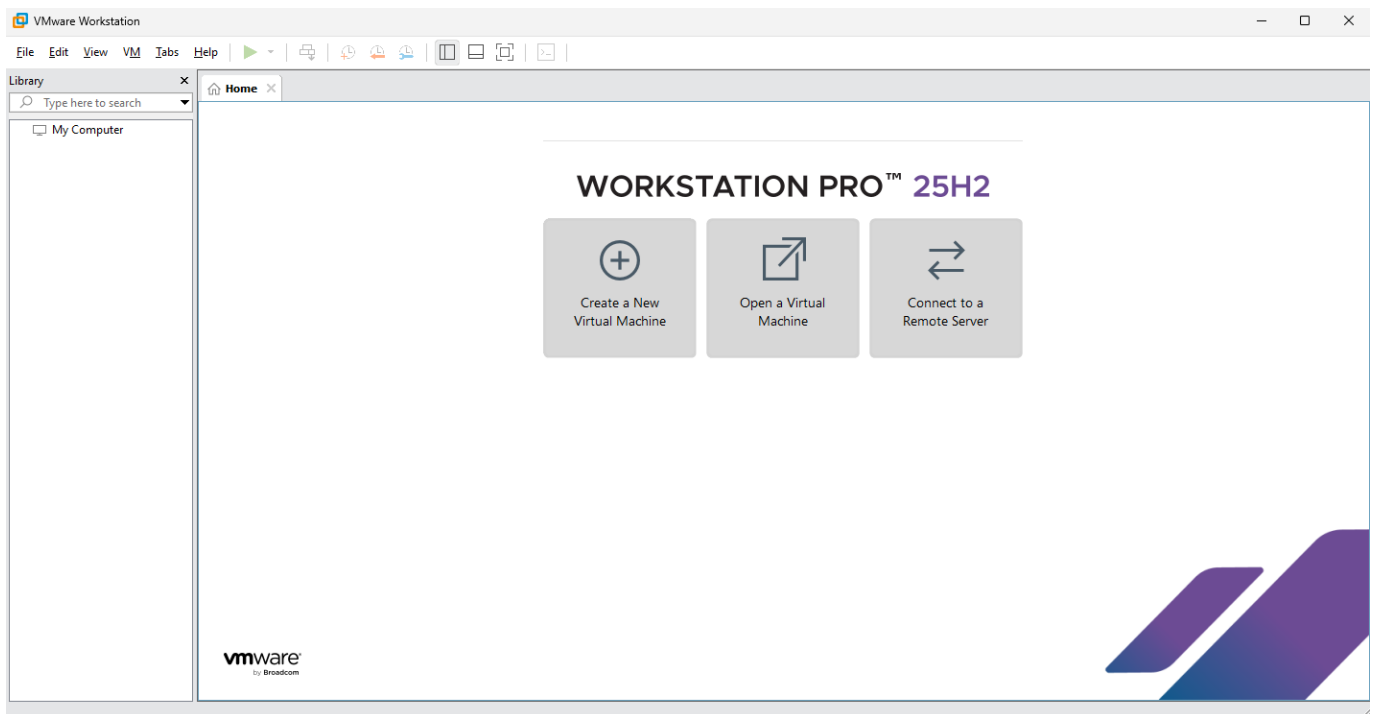


8. Restart the System

- Restart your computer if prompted
- This ensures proper setup of drivers

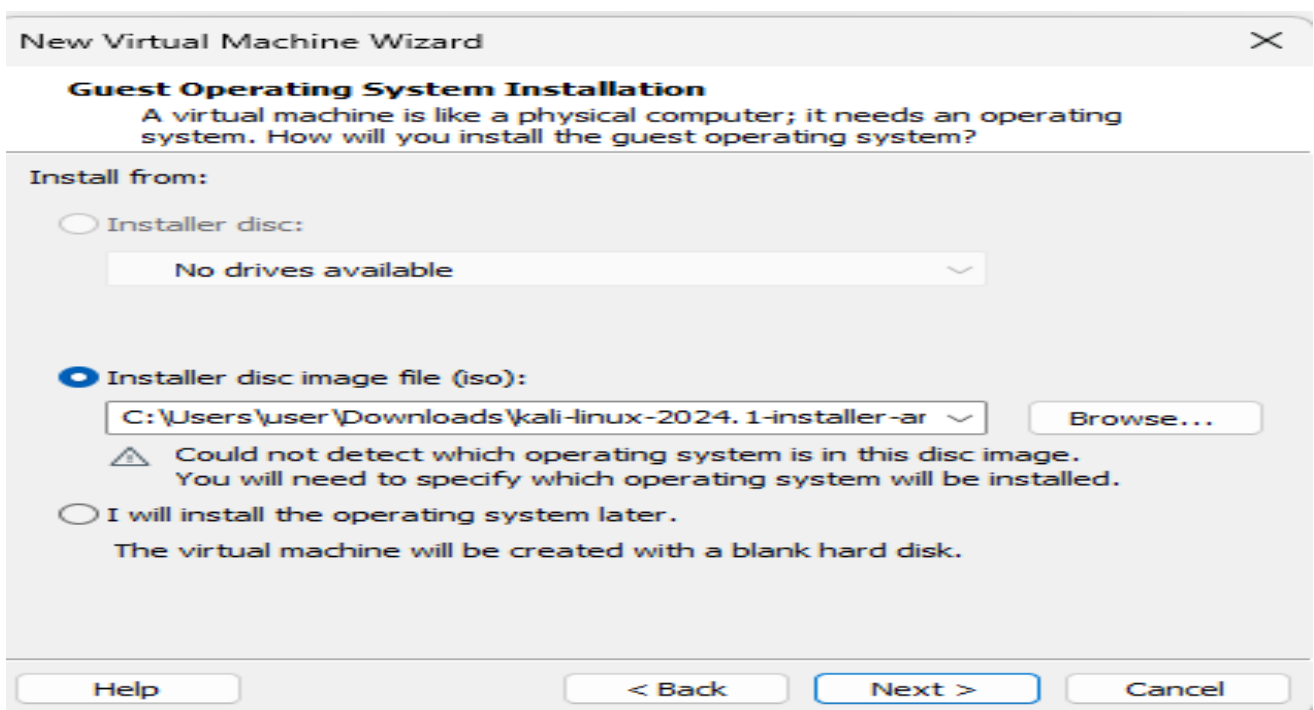
9. Launch VMware

- Open VMware Workstation Player from Desktop or Start Menu
- Select **Use for non-commercial use**
- Click **Continue**



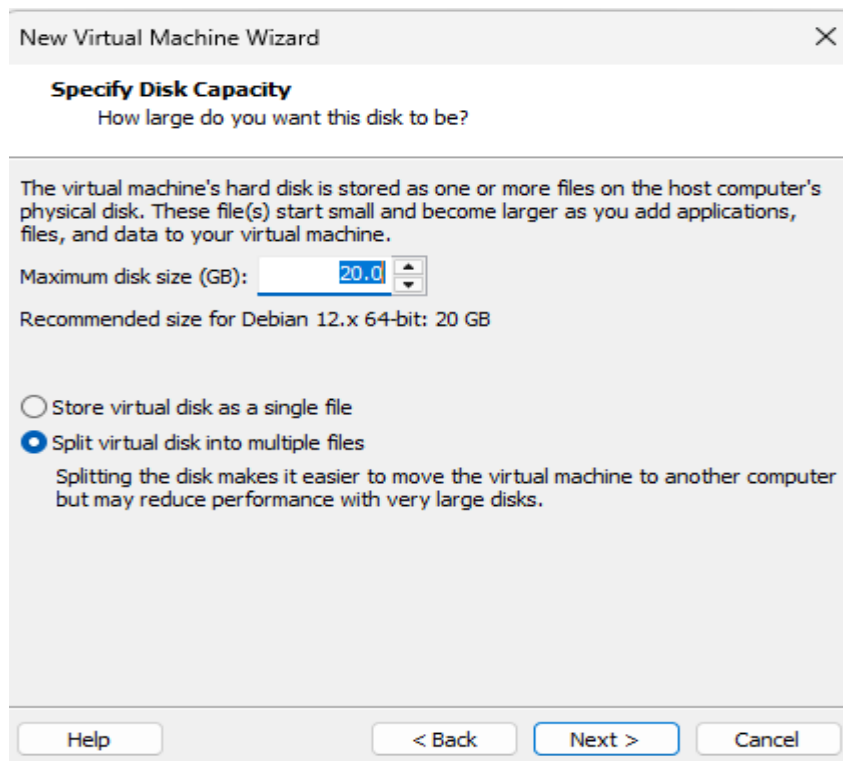
10. Create a New Virtual Machine

- Click on **Create a New Virtual Machine**
- Choose installation method:
 - ISO image file (recommended)
 - Physical disk (optional)
- Browse and select the OS ISO file



11. Configure Virtual Machine

- Enter name and location of VM
- Allocate disk size (20 GB recommended)
- Customize hardware (RAM, CPU if needed)



The screenshot shows a window titled "New Virtual Machine Wizard" with a close button (X) in the top right corner. The main heading is "Specify Disk Capacity" followed by the question "How large do you want this disk to be?". Below this, a text box explains: "The virtual machine's hard disk is stored as one or more files on the host computer's physical disk. These file(s) start small and become larger as you add applications, files, and data to your virtual machine." A label "Maximum disk size (GB):" is followed by a text input field containing "20.0" and a spinner control. Below the input field, it says "Recommended size for Debian 12.x 64-bit: 20 GB". There are two radio button options: "Store virtual disk as a single file" (unselected) and "Split virtual disk into multiple files" (selected). A note below the selected option states: "Splitting the disk makes it easier to move the virtual machine to another computer but may reduce performance with very large disks." At the bottom, there are four buttons: "Help", "< Back", "Next >" (highlighted with a blue border), and "Cancel".

12. Complete Setup

- Click **Finish**
- Start the virtual machine
- Follow OS installation steps inside VM

Practical - 9

Aim:- To execute any 10 Hadoop commands on VmWare.

1. pwd – Print Working Directory

Shows your current directory.

pwd

```
(sahilgupta@kali)-[~]  
$ pwd  
/home/sahilgupta
```

2. ls – List Files

Lists files and folders in the current directory.

ls

```
(sahilgupta@kali)-[~]  
$ ls  
Desktop Documents Downloads file1.txt hadoop hive metastore_db Music Pictures pig Public Templates Videos
```

3. cd – Change Directory

Move between directories.

cd /home

```
(sahilgupta@kali)-[/]  
$ cd /home  
  
(sahilgupta@kali)-[/home]  
$
```

4. nano – Create File

Creates a new empty file.

nano file1.txt

```
(sahilgupta@kali)-[/home]  
$ nano file1.txt
```

5. echo – Print Text / Write to File

echo "Hello Linux" > file1.txt

```
(sahilgupta@kali)-[~]  
$ echo "Hello LINUX"> file1.txt
```

6. cat – View File Content

cat file1.txt

```
(sahilgupta@kali)-[~]  
$ cat file1.txt  
Hello Hadoop
```

7. rm – Remove File

rm file1.txt

```
(sahilgupta@kali)-[~]  
$ rm file1.txt  
  
(sahilgupta@kali)-[~]  
$ ls  
Desktop Documents Downloads hadoop hive metastore_db Music Pictures pig Public Templates Videos
```

8. mkdir – Create Directory

mkdir testdir

```
(sahilgupta@kali)-[~]  
$ mkdir testdir  
  
(sahilgupta@kali)-[~]  
$ ls  
Desktop Documents Downloads hadoop hive metastore_db Music Pictures pig Public Templates testdir Videos
```

9. rmdir – Remove Directory

rmdir testdir

```
(sahilgupta@kali)-[~]  
$ rmdir testdir  
  
(sahilgupta@kali)-[~]  
$ ls  
Desktop Documents Downloads hadoop hive metastore_db Music Pictures pig Public Templates Videos
```

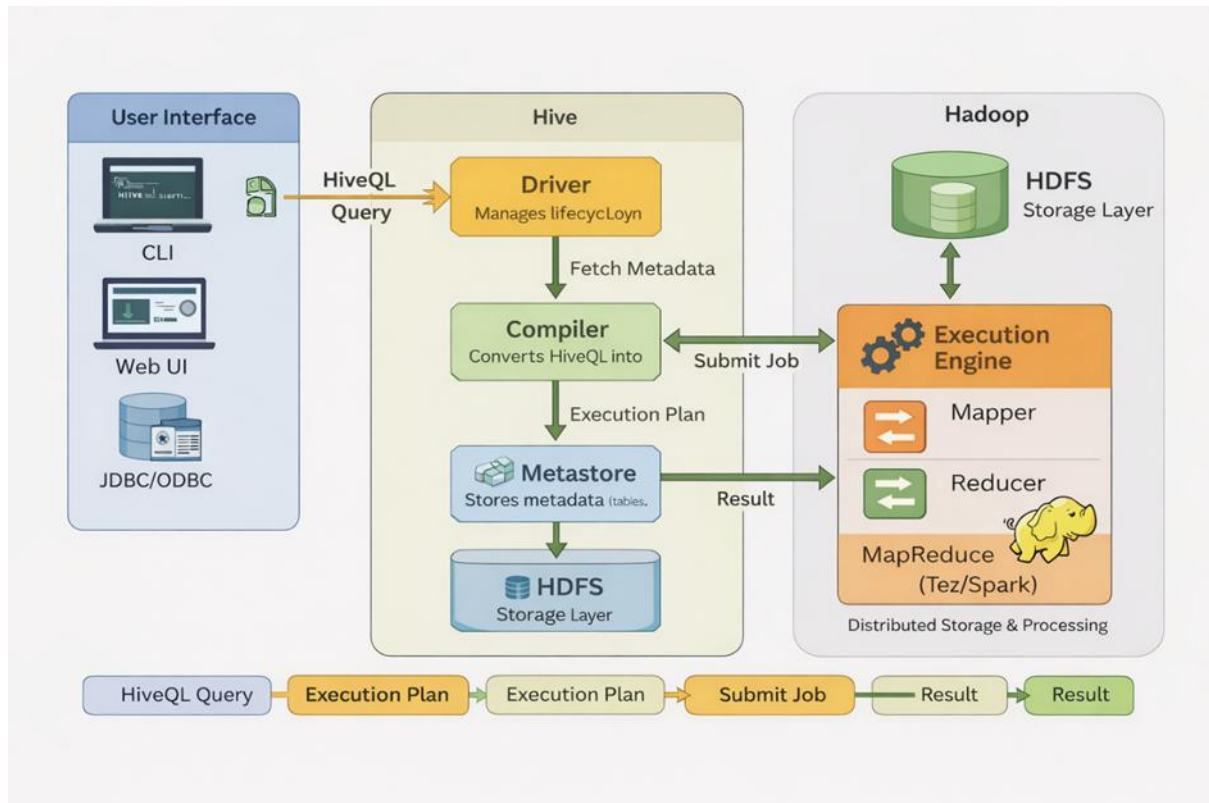
10. whoami – Current User

Whoami

```
(sahilgupta@kali)-[~]  
$ whoami  
sahilgupta
```

Practical 10

AIM: Introduction to Hive and pig data module.



Apache Hive

Apache Hive is a **data warehouse system built on Hadoop** that enables users to query and analyze large datasets using a SQL-like language known as **HiveQL**. It makes data processing easier by removing the need to write complex MapReduce programs.

Hive Architecture

- **User Interface** – Provides interaction through CLI, Web UI, JDBC/ODBC
- **Driver** – Handles the query execution process and manages sessions
- **Compiler** – Converts HiveQL queries into an execution plan
- **Metastore** – Stores metadata such as table structure and schema
- **Execution Engine** – Runs queries by interacting with Hadoop components
- **HDFS** – Acts as the storage layer where data is stored

Flow

HiveQL → Compiler → Execution Engine → Hadoop → Output Result

Installation of Hive (Basic Steps)

1. Ensure Hadoop is properly installed and running
2. Download Hive using:
wget <https://downloads.apache.org/hive/>

3. Extract the downloaded package:
`tar -xvf apache-hive-*.tar.gz`
4. Set environment variables:
`export HIVE_HOME=/path/to/hive`
`export PATH=$PATH:$HIVE_HOME/bin`
5. Configure the metastore (Derby or MySQL)
6. Initialize the Hive schema:
`schematool -initSchema -dbType derby`
7. Start Hive using:
`hive`

Hive vs RDBMS

S.No	Apache Hive	RDBMS(MySQL or Oracle or SQL Server, etc.)
1	SQL interface supported	SQL interface supported
2	Focus on analytics and big data analytics(OLAP, Batch Processing)	Focus on online or analytics (OLTP, OLAP)
3	Limited transactions support*	Transactions are usually support
4	Distributed processing via MapReduce	Distributed processing varies by vendor(if available)
5	Scale to hundreds of nodes	It has limitation for scaling the nodes
6	Built on commodity hardware	Often built on proprietary/specialized hardware(especially when scaling out)
7	Limited indexing support*	It has full indexing support
8	No triggers	It has triggers support

Apache Pig

Introduction

Apache Pig is a **high-level data processing framework** built on Hadoop, used for handling and analyzing large datasets. It uses a scripting language called **Pig Latin**, which makes complex data operations much easier compared to writing MapReduce programs.



Why Pig?

- Simplifies complex data processing tasks compared to MapReduce
- Uses an easy and readable scripting language (Pig Latin)
- Suitable for processing huge volumes of data
- Reduces coding effort and development time

Pig Architecture

- **Pig Latin Script** – Script written by the user
- **Parser** – Validates syntax and checks errors
- **Logical Plan** – Represents the flow of data operations
- **Optimizer** – Enhances query performance
- **Compiler** – Converts script into MapReduce jobs
- **Execution Engine** – Executes jobs on Hadoop

Pig Data Model

1. **Atom** – Represents a single data value (e.g., 5, "Data")

2. **Tuple** – Collection of fields arranged in order
o Example: (2, 'Rahul', 25)
3. **Bag** – Group of tuples
o Example: {(1,'X'), (2,'Y')}
4. **Map** – Collection of key-value pairs
o Example: [name#'Rahul', age#25]

Installation of Pig (Basic Steps)

1. Ensure Hadoop is installed and configured
2. Download Pig using:
wget <https://downloads.apache.org/pig/>
3. Extract the downloaded package:
tar -xvf pig-*.tar.gz
4. Set environment variables:
export PIG_HOME=/your/pig/path
export PATH=\$PATH:\$PIG_HOME/bin
5. Start Pig using:
pig

HQL vs SQL

HQL Hive Query Language	SQL Structured Query Language
Used in Apache Hive to query and analyze data stored in Hadoop	Standard language for managing and querying data in databases like MySQL, Oracle
Transforms HQL queries into Hadoop jobs (MapReduce)	Directly interacts with the database
Best for big data and batch processing	Works best for transactional data
Limited support for transactions	Strong support for transactions
Optimized for distributed data processing	Optimized for quick data retrieval
Optimized for distributed data processing	Optimized for quick data retrieval

Practical 11

AIM: Write a program to read and store the data using PIG grunt shell.

Execute the diagnostic operators on this data.

Step 1: Sample Data File

Create a file data.txt:

```
1,Sahil,22
2,SHivansh,22
3,Abhishek Sheji,23
4,Abhishek,25
```

Step 2: Start Pig Grunt Shell

```
pig -x local
```

Step 3: Load the Data

```
data = LOAD 'data.txt' USING PigStorage(',')
      AS (id:int, name:chararray, age:int);
```

Step 4: Diagnostic Operators

These operators help **debug** and **inspect data**.

```
DESCRIBE data; (Schema view)
```

```
data: {id: int,name: chararray,age: int}
```

```
DUMP data; (Display data)
```

```
(1,Sahil,22)
(2,SHivansh,22)
(3,Abhishek Sheji,23)
(4,Abhishek,25)
```

```
EXPLAIN data; (Execution plan)
```

Displays how data flows through operations.

Practical 12

AIM: Write any 10 HIVE DDL commands and HIVE DML commands.

HIVE DDL Commands (Data Definition Language)

These commands define or modify database structure.

1. Create Database

```
CREATE DATABASE company_db;
```

```
hive> CREATE DATABASE company_db;  
OK  
Time taken: 0.67 seconds
```

2. Show Databases

```
SHOW DATABASES;
```

```
hive> SHOW DATABASES;  
OK  
company_db  
default  
Time taken: 0.249 seconds, Fetched: 2 row(s)
```

3. Use Database

```
USE company_db;
```

```
hive> USE company_db;  
OK  
Time taken: 0.07 seconds
```

4. Create Table

```
CREATE TABLE employees (  
    id INT,  
    name STRING,  
    salary FLOAT  
);
```

```
hive> CREATE TABLE employees (  
    > id INT,  
    > name STRING,  
    > salary FLOAT  
    > )  
    > ROW FORMAT DELIMITED  
    > FIELDS TERMINATED BY ',';  
OK  
Time taken: 0.771 seconds
```

5. Describe Table

```
DESCRIBE employees;
```

```
hive> DESCRIBE employees;
OK
id                int
name              string
salary           float
Time taken: 0.256 seconds, Fetched: 3 row(s)
```

6. Alter Table (Add Column)

```
ALTER TABLE employees ADD COLUMNS (age INT);
```

```
hive> ALTER TABLE employees ADD COLUMNS (age INT);
OK
Time taken: 0.208 seconds
```

7. Rename Table

```
ALTER TABLE employees RENAME TO staff;
```

```
hive> ALTER TABLE employees RENAME TO staff;
OK
Time taken: 0.187 seconds
```

8. Drop Table

```
DROP TABLE staff;
```

```
hive> DROP TABLE staff;
OK
Time taken: 0.518 seconds
```

9. Truncate Table

```
TRUNCATE TABLE employees;
```

```
hive> TRUNCATE TABLE employees;
OK
Time taken: 0.293 seconds
```

10. Drop Database

```
DROP DATABASE company_db;
```

```
hive> DROP DATABASE company_db;
OK
Time taken: 0.24 seconds
```

HIVE DML Commands (Data Manipulation Language)

These commands deal with data inside tables.

1. Load Data

```
LOAD DATA LOCAL INPATH 'employees.csv' INTO TABLE employees;
```

```
hive> LOAD DATA LOCAL INPATH '/home/sahilgupta/emp.txt'
> INTO TABLE employees;
Loading data to table company_db.employees
OK
Time taken: 1.094 seconds
```

2. Select Data

```
SELECT * FROM employees;
```

```
hive> SELECT * FROM employees;
OK
1      John      50000.0
2      Alice     60000.0
Time taken: 2.398 seconds, Fetched: 2 row(s)
```

3. Select with Condition

```
SELECT * FROM employees WHERE salary > 40000;
```

```
hive> SELECT * FROM employees WHERE salary > 40000;
OK
1      John      50000.0
2      Alice     60000.0
Time taken: 0.63 seconds, Fetched: 2 row(s)
```

4. Insert from Another Table

```
INSERT INTO employees SELECT * FROM new_employees;
```

```
hive> INSERT OVERWRITE TABLE employees
> SELECT * FROM temp_employees;
Query ID = sahilgupta_20260422124606_70a0e6be-f43c-42f2-84d6-0ad0d03d3d
18
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-04-22 12:46:10,319 Stage-1 map = 0%, reduce = 0%
Ended Job = job_local1904576177_0001 with errors
Error during job, obtaining debugging information...
FAILED: Execution Error, return code 2 from org.apache.hadoop.hive.ql.e
xec.mr.MapRedTask
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 0 HDFS Write: 0 FAIL
Total MapReduce CPU Time Spent: 0 msec
```

5. Create Table as Select (CTAS)

```
CREATE TABLE high_salary AS
SELECT * FROM employees WHERE salary > 60000;
```

```
hive> CREATE TABLE high_salary AS
> SELECT * FROM employees
> WHERE salary > 50000;
Query ID = sahilgupta_20260422124755_c987ae8d-cd99-4cff-825c-1f7a81eecd
8f
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks is set to 0 since there's no reduce operator
Job running in-process (local Hadoop)
2026-04-22 12:47:57,305 Stage-1 map = 0%, reduce = 0%
2026-04-22 12:47:58,338 Stage-1 map = 100%, reduce = 0%
Ended Job = job_local1099542680_0002
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/comp
any_db.db/.hive-staging_hive_2026-04-22_12-47-55_751_821926485258274649
9-1/-ext-10002
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/comp
any_db.db/high_salary
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 81 HDFS Write: 148 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
Time taken: 2.921 seconds
```

6. Delete (Replaced with INSERT OVERWRITE)

```
INSERT OVERWRITE TABLE employees
SELECT * FROM employees WHERE id != 1;
```

```

hive> INSERT OVERWRITE TABLE employees
> SELECT * FROM employees
> WHERE id ≠ 1;
Query ID = sahilgupta_20260422124850_f3ccad98-ba6d-4cbe-b344-a26ad325e55e
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-04-22 12:48:51,677 Stage-1 map = 0%, reduce = 0%
Ended Job = job_local552350962_0003 with errors
Error during job, obtaining debugging information...
FAILED: Execution Error, return code 2 from org.apache.hadoop.hive.ql.exec.mr.MapRedTask
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 0 HDFS Write: 0 FAIL
Total MapReduce CPU Time Spent: 0 msec

```

7. Update (Replaced with INSERT OVERWRITE + CASE)

```

INSERT OVERWRITE TABLE employees
SELECT
  id,
  name,
  CASE
    WHEN id = 2 THEN 55000
    ELSE salary
  END AS salary
FROM employees;

```

```

hive> INSERT OVERWRITE TABLE employees
> SELECT
>   id,
>   name,
>   CASE
>     WHEN id = 2 THEN 70000
>     ELSE salary
>   END AS salary
> FROM employees;
Query ID = sahilgupta_20260422124929_e0f4a83b-d6cd-4e62-91a7-99054235f550
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-04-22 12:49:30,740 Stage-1 map = 0%, reduce = 0%
Ended Job = job_local2087897532_0004 with errors
Error during job, obtaining debugging information...
FAILED: Execution Error, return code 2 from org.apache.hadoop.hive.ql.exec.mr.MapRedTask
MapReduce Jobs Launched:
Stage-Stage-1:  HDFS Read: 0 HDFS Write: 0 FAIL
Total MapReduce CPU Time Spent: 0 msec

```

8. Export Data

```
INSERT OVERWRITE DIRECTORY '/output/employees'  
SELECT * FROM employees;
```

```
hive> INSERT OVERWRITE DIRECTORY '/home/sahilgupta/output_hive'  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > SELECT * FROM employees;  
Query ID = sahilgupta_20260422125036_cab75c89-cf36-4cb1-ae05-ca310bf4b6d6  
Total jobs = 3  
Launching Job 1 out of 3  
Number of reduce tasks is set to 0 since there's no reduce operator  
Job running in-process (local Hadoop)  
2026-04-22 12:50:38,098 Stage-1 map = 100%, reduce = 0%  
Ended Job = job_local2017070743_0005  
Stage-3 is selected by condition resolver.  
Stage-2 is filtered out by condition resolver.  
Stage-4 is filtered out by condition resolver.  
Moving data to directory hdfs://localhost:9000/home/sahilgupta/output_hive/.hive-staging_hive_2026-04-22_12-50-36_591_961334237674330985-1/-ext-10000  
Moving data to directory /home/sahilgupta/output_hive  
MapReduce Jobs Launched:  
Stage-Stage-1:  HDFS Read: 186 HDFS Write: 179 SUCCESS  
Total MapReduce CPU Time Spent: 0 msec  
OK  
Time taken: 1.895 seconds
```

Practical 13

AIM: Write a HQL command to create a table employee with your own attribute and execute the following command

- SELECT all fields
- GROUP BY department
- WHERE any condition
- HAVING
- ORDER BY employee number

1. Create Table

```
CREATE TABLE employee (  
    emp_id INT,  
    emp_name STRING,  
    department STRING,  
    salary DOUBLE,  
    age INT  
)  
ROW FORMAT DELIMITED  
FIELDS TERMINATED BY ','  
STORED AS TEXTFILE;
```

```
hive> CREATE TABLE employee (  
    > emp_id INT,  
    > emp_name STRING,  
    > department STRING,  
    > salary DOUBLE,  
    > age INT  
    > )  
    > ROW FORMAT DELIMITED  
    > FIELDS TERMINATED BY ','  
    > STORED AS TEXTFILE;  
OK  
Time taken: 0.103 seconds
```

2. SELECT all fields

```
SELECT * FROM employee;
```

```
hive> SELECT * FROM employee;  
OK  
101    Amit    HR        35000.0 25  
102    Neha    IT        50000.0 28  
103    Raj     IT        45000.0 30  
104    Priya   Finance  40000.0 27  
105    Karan   HR        30000.0 26  
Time taken: 0.198 seconds, Fetched: 5 row(s)
```

3. WHERE condition

```
SELECT *  
FROM employee  
WHERE salary > 30000;
```

```
hive> SELECT *  
  > FROM employee  
  > WHERE salary > 30000;  
OK  
101    Amit    HR      35000.0 25  
102    Neha    IT      50000.0 28  
103    Raj     IT      45000.0 30  
104    Priya   Finance 40000.0 27  
Time taken: 0.171 seconds, Fetched: 4 row(s)
```

4. GROUP BY department

```
SELECT department, AVG(salary) AS avg_salary  
FROM employee  
GROUP BY department;
```

```
hive> SELECT department, AVG(salary) AS avg_salary  
  > FROM employee  
  > GROUP BY department;  
Query ID = arya_20260403103717_ec19fbd6-59dd-41a3-8525-f1616057afca  
Total jobs = 1  
Launching Job 1 out of 1  
Number of reduce tasks not specified. Estimated from input data size: 1  
In order to change the average load for a reducer (in bytes):  
  set hive.exec.reducers.bytes.per.reducer=<number>  
In order to limit the maximum number of reducers:  
  set hive.exec.reducers.max=<number>  
In order to set a constant number of reducers:  
  set mapreduce.job.reduces=<number>  
Starting Job = job_1775192426488_0002, Tracking URL = http://Aryan.localdomain:8088/proxy/application_1775192426488_0002/  
Kill Command = /usr/local/hadoop/bin/mapred job -kill job_1775192426488_0002  
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1  
2026-04-03 10:37:26,778 Stage-1 map = 0%, reduce = 0%  
2026-04-03 10:37:31,914 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 4.89 sec  
2026-04-03 10:37:38,089 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 7.59 sec  
MapReduce Total cumulative CPU time: 7 seconds 590 msec  
Ended Job = job_1775192426488_0002  
MapReduce Jobs Launched:  
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 7.59 sec HDFS Read: 15746 HDFS Write: 161 SUCCESS  
Total MapReduce CPU Time Spent: 7 seconds 590 msec  
OK  
Finance 40000.0  
HR      32500.0  
IT      47500.0  
Time taken: 22.887 seconds, Fetched: 3 row(s)
```

5. HAVING clause

```
SELECT department, AVG(salary) AS avg_salary
FROM employee
GROUP BY department
HAVING AVG(salary) > 40000;
```

```
hive> SELECT department, AVG(salary) AS avg_salary
> FROM employee
> GROUP BY department
> HAVING AVG(salary) > 40000;
Query ID = arya_20260403103758_02837691-6081-422e-992f-cefd7d59cbc0
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1775192426488_0003, Tracking URL = http://Aryan.localdomain:8088/proxy/application_1775192426488_0003/
Kill Command = /usr/local/hadoop/bin/mapred job -kill job_1775192426488_0003
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-03 10:38:07,810 Stage-1 map = 0%, reduce = 0%
2026-04-03 10:38:12,936 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 3.6 sec
2026-04-03 10:38:18,051 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 6.1 sec
MapReduce Total cumulative CPU time: 6 seconds 100 msec
Ended Job = job_1775192426488_0003
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 6.1 sec HDFS Read: 16324 HDFS Write: 110 SUCCESS
Total MapReduce CPU Time Spent: 6 seconds 100 msec
OK
IT 47500.0
Time taken: 21.262 seconds, Fetched: 1 row(s)
```

6. ORDER BY employee number

```
SELECT *
FROM employee
ORDER BY emp_id;
```

```
hive> SELECT *
> FROM employee
> ORDER BY emp_id;
Query ID = arya_20260403103826_069a44c1-4b4c-42f3-a431-9928933c2e23
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1775192426488_0004, Tracking URL = http://Aryan.localdomain:8088/proxy/application_1775192426488_0004/
Kill Command = /usr/local/hadoop/bin/mapred job -kill job_1775192426488_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-03 10:38:35,841 Stage-1 map = 0%, reduce = 0%
2026-04-03 10:38:40,967 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 3.43 sec
2026-04-03 10:38:45,068 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 5.15 sec
MapReduce Total cumulative CPU time: 5 seconds 150 msec
Ended Job = job_1775192426488_0004
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 5.15 sec HDFS Read: 11937 HDFS Write: 268 SUCCESS
Total MapReduce CPU Time Spent: 5 seconds 150 msec
OK
101 Amit HR 35000.0 25
102 Neha IT 50000.0 28
103 Raj IT 45000.0 30
104 Priya Finance 40000.0 27
105 Karan HR 30000.0 26
Time taken: 20.791 seconds, Fetched: 5 row(s)
```

Practical 14

AIM: To perform data analysis using Sqoop.

Using **Apache Sqoop**, you don't actually *perform* data analysis directly—Sqoop's main job is to **transfer data between relational databases and Hadoop ecosystems** (like HDFS, Hive, or HBase).

The *analysis* part is usually done afterward using tools like Hive, Spark, or MapReduce.

Here's a clear breakdown of how Sqoop fits into a data analysis workflow:

What Sqoop Does in Data Analysis

Sqoop is used for:

- Importing data from RDBMS (MySQL, Oracle, PostgreSQL) → Hadoop (HDFS/Hive/HBase)
- Exporting processed data back to databases
- Automating bulk data transfer

Think of Sqoop as the **bridge** between structured databases and big data tools.

Typical Data Analysis Workflow Using Sqoop

1. Import Data into Hadoop

You bring data from a database into HDFS or Hive.

Example command (MySQL → HDFS):

```
sqoop import \  
--connect jdbc:mysql://localhost/sales_db \  
--username root \  
--password pass123 \  
--table orders \  
--target-dir /data/orders \  
--num-mappers 4
```

What happens:

- Data is split across multiple mappers
- Stored in HDFS for distributed processing

2. Import Data Directly into Hive (for Analysis)

```
sqoop import \  
--connect jdbc:mysql://localhost/sales_db \  
--username root \  
--password pass123 \  
--table orders \  
--hive-import \  
--create-hive-table \  
--hive-table orders_hive
```

Now you can run SQL-like queries in Hive.

3. Perform Data Analysis (Using Hive/Spark)

Example Hive query:

```
SELECT product_id, SUM(amount)  
FROM orders_hive  
GROUP BY product_id;
```

4. Export Results Back to Database (Optional)

```
sqoop export \  
--connect jdbc:mysql://localhost/sales_db \  
--username root \  
--password pass123 \  
--table summary \  
--export-dir /user/hive/warehouse/result
```

When to Use Sqoop

Use Sqoop when:

- You have structured data in databases
- You want to analyze it using big data tools
- You need bulk transfer efficiently

Limitations

- Not used for real-time data (use Kafka instead)
- Not for analysis itself
- Requires Hadoop ecosystem